# Code Review & Refactor Agent

An AI-powered multi-agent system that analyzes GitHub pull requests, generates refactoring suggestions with parallel test and security analysis, evaluates them with LLM-as-Judge, and applies patches upon human approval.

## What It Does

When a pull request is opened, the agent fetches all changed files, creates a prioritized task plan, performs deep AST and LLM-based code analysis, and proposes refactored implementations. Test and security agents run in parallel to validate the proposed changes. A review node merges all findings into a final implementation, which is evaluated by an LLM-as-Judge before being presented to the developer for approval. Upon approval, the agent commits the patch and posts inline review comments to the PR.

## Architecture

```
START
  → supervisor        # fetch PR files, create task plan, detect
injection
  → code_analyst      # AST analysis, dependency mapping, pattern
detection
  → implementation    # propose refactoring based on analysis
  → [test_agent || security_agent]   # parallel: unit tests + security
scan
  → review            # merge findings, revise implementation
  → judge             # LLM-as-Judge quality evaluation (score 0–1)
  → human_review      # HITL: pause and wait for developer approval
  → pr_agent          # apply patch, post PR description and inline
comments
  → END
```

`test_agent` and `security_agent` run in parallel via LangGraph's fan-out/fan-in edge pattern. `review` waits for both to complete before merging.

## Tech Stack

| Tool | Purpose |
|---|---|
| LangGraph | Agent orchestration — state, parallel edges, HITL interrupt |
| LiteLLM | Unified LLM interface — model switching via `MODEL_NAME` env var |
| LangSmith | Tracing — node-level inputs, outputs, and latency |

| Tool | Purpose |
|---|---|
| FastAPI | Webhook server — GitHub PR events and approval endpoint |
| slowapi | Rate limiting — 10 requests/minute on `/webhook` |
| PyGithub | GitHub API — fetch PR files, apply patches, post comments |
| Pydantic | Schema validation — structured LLM output and webhook payloads |

## Project Structure

```
code-review-agent/
├── agent/
│   ├── graph.py          # LangGraph graph definition and compilation
│   ├── nodes.py          # All 9 async node functions
│   ├── state.py          # PRReviewState TypedDict and sub-types
│   └── tools/
│       ├── ast_parser.py # Python AST analysis
│       ├── git_patch.py  # GitHub API: fetch files, apply patch, post
comments
│       ├── security.py   # Prompt injection detection + AST/LLM
validation
│       └── utils.py      # call_llm_with_retry, parse_llm_json
├── api/
│   ├── main.py           # FastAPI app — logging, rate limiter, body
size limit
│   ├── webhook.py        # /webhook and /approve endpoints
│   └── schemas.py        # Pydantic schemas for LLM output and webhook
payload
├── config/
│   ├── prompts.py        # All LLM prompt functions with <code_diff>
isolation
│   └── settings.py       # Environment variable loading and validation
├── eval/
│   └── judge.py          # LLM-as-Judge evaluation function
├── tests/
│   ├── conftest.py
│   ├── test_nodes.py
│   ├── test_judge.py
│   ├── test_utils.py
│   ├── test_security.py
│   ├── test_git_patch.py
│   └── test_ast_parser.py
├── docs/                 # Architecture Decision Records (ADR-001 to
ADR-012)
├── .github/workflows/
│   └── ci.yml            # CI pipeline: test → lint → build
├── Dockerfile
├── docker-compose.yml
└── .env.example
```

## Setup

### Requirements

- Python 3.11+
- Docker (optional)

### Installation

```
git clone <repo-url>
cd code-review-agent
pip install -r requirements.txt
```

### Environment Variables

Copy `.env.example` and fill in the values:

```
cp .env.example .env
```

| Variable | Required | Description |
|---|---|---|
| `GITHUB_TOKEN` | Yes | GitHub personal access token with repo scope |
| `WEBHOOK_SECRET` | Recommended | Secret for HMAC-SHA256 webhook signature verification |
| `OPENAI_API_KEY` | Yes | OpenAI API key (or the key for your chosen provider) |
| `MODEL_NAME` | No | LLM model to use (default: `gpt-4o-mini`) |
| `LANGCHAIN_API_KEY` | No | LangSmith API key for tracing |
| `LANGCHAIN_PROJECT` | No | LangSmith project name |
| `LANGCHAIN_TRACING_V2` | No | Set to `true` to enable tracing |

## Running

### With Docker

```
docker compose up --build
```

### Manually

```
uvicorn api.main:app --reload
```

The server starts on `http://localhost:8000`.

---

## Usage

### 1. Configure GitHub webhook

In your GitHub repository settings, add a webhook:

- **Payload URL:** `https://your-server/webhook`
- **Content type:** `application/json`
- **Secret:** value of `WEBHOOK_SECRET`
- **Events:** Pull requests

### 2. Trigger a review

When a PR is opened or updated, GitHub sends a webhook. The agent processes the PR and returns:

```
{
  "thread_id": "abc-123",
  "pr_id": 42,
  "judge_scores": [{"file_name": "foo.py", "score": 0.9, "reasoning":
"..."}],
  "judge_status": "passed",
  "pr_review_steps": ["Supervisor: task plan created.", "..."]
}
```

### 3. Approve or reject

```
POST /approve?thread_id=abc-123&approved=true
```

On approval, the agent applies the patch and posts inline review comments. On rejection, the workflow ends without touching the branch.

---

## Security

The agent implements a four-layer defence strategy:

| Layer | Mechanism |
|---|---|
| 1 — Webhook signature | HMAC-SHA256 verification of `X-Hub-Signature-256` header |

| Layer | Mechanism |
|---|---|
| 2 — Prompt injection detection | Regex scan of code diffs before any LLM call |
| 3 — Prompt delimiter | `<code_diff>` tags isolate user code from LLM instructions |
| 4 — Output validation | AST findings override LLM output on numeric/severity conflicts |

Additionally: rate limiting (10 req/min on `/webhook`) and 1 MB request body limit.

---

## How HITL Works

When the workflow reaches `human_review_node`, LangGraph's `interrupt()` pauses execution and saves the full `PRReviewState` to `MemorySaver`. The `/webhook` response includes the `thread_id`. The developer reviews `judge_scores` and `final_review`, then calls `/approve` with the `thread_id`. FastAPI resumes the graph via `graph.ainvoke(Command(resume=approved), config)`.

---

## LLM-as-Judge

`judge_node` runs after `review_node` and before `human_review_node`. It compares the original code against the revised implementation and returns a `score` (0–1) with `reasoning`. Scores below 0.5 set `judge_status` to `"warning"`. The developer sees both the score and reasoning before deciding to approve.

---

## Error Handling

| Failure | Strategy |
|---|---|
| LLM transient error | Exponential backoff retry (max 3 attempts, 2s/4s waits) |
| LLM JSON parse error | Retry with stricter prompt — no backoff delay |
| GitHub API error | Descriptive `RuntimeError` with HTTP status and message |
| AST parse failure | Skip file with `logger.warning`, continue with other files |
| Prompt injection detected | Skip infected file with warning, continue with clean files |
| Missing credentials | `EnvironmentError` at startup — fails fast before workflow begins |
| Request body > 1 MB | HTTP 413 from middleware before any processing |
| Invalid webhook signature | HTTP 401 from `Depends` before payload is parsed |

---

## Testing

```
pytest
```

All tests are async (`pytest-asyncio`, `asyncio_mode = "auto"`). External dependencies — LLM API, GitHub API — are mocked. Tests run without real credentials and without making network calls.

```
tests/
├── test_nodes.py       # 9 node functions, including injection-skip
behaviour
├── test_judge.py       # async judge_code_refactory
├── test_utils.py       # call_llm_with_retry: backoff, strict prompt,
parse helpers
├── test_security.py    # detect_prompt_injection, validate_llm_output
├── test_git_patch.py   # get_pr_files, apply_patch, create_pr_comment
└── test_ast_parser.py  # analyze_code
```

---

## Architecture Decisions

See the `docs/` directory for 12 Architecture Decision Records (ADR-001 to ADR-012) covering choices such as LangGraph over LCEL, LLM-as-Judge placement, retry strategy, and all four security layers.

---

## Known Limitations

- AST analysis supports Python files only
- `MemorySaver` stores state in process memory — restarting the server loses all pending `thread_id` states. A persistent checkpointer (PostgreSQL, Redis) is required for production
- Regex-based injection detection produces false positives on legitimate code that uses common phrases in comments or docstrings
- `MODEL_NAME` applies globally — all nodes use the same model. Per-node model configuration is not supported

---